Learned Layers for a Factor-Based PDE Pipeline: What Worked, What Did Not, and Why

Flavio Pricoco*

November 9, 2026

Abstract. A pipeline that reads the factors of a linear partial differential equation from noisy data already contains classical statistical learning: a Gaussian process interpolates, a Dirichlet-process mixture groups, a Bayesian linear model splits components. This paper asks a different question: can *learned* layers, trained on simulated equations, make that pipeline more accurate, better calibrated or faster, without ever changing what it is allowed to claim? We state a contract under which a learned layer may propose a candidate, fit a global constant, or raise a flag, but never decide a label, and under which every layer switched off reproduces the pipeline bit for bit. We designed eight such layers, built seven, and gated each on held-out equation families, with leave-one-family-out validation, paired cluster bootstrap and conformal operating points. One layer ships: an exact memo that is bit-identical to the pipeline and saves about half a percent of the time. A learned failure detector separates injected failures from clean runs (cross-validated AUROC 0.85, against 0.63 for the pipeline's own omnibus test) but has almost no detection power at the 5% false-alarm level it needs, and a controller that reruns the pipeline with other settings has an oracle ceiling of 8 of 12 failures, reduced to 0 of 12 once a real trigger is required. The calibration layers fail because the quantity they would calibrate is saturated or already calibrated. We report each negative result with its numbers, the lessons they share, and the ideas that remain.

Key words. equation discovery, uncertainty calibration, conformal prediction, failure detection, negative results

MSC codes. 35E20, 62F15, 62G15, 68T05

1 Introduction

The pipeline studied here takes noisy observations of a field $u(t, x)$ that solves a linear constant-coefficient equation and returns a distribution over equations, each with intervals on its coefficients and one of four labels: FIT-COMPATIBLE, MISFIT, ID-KER, UNDECIDED. It never prints that a law is identified. Its design and the mathematics behind it are in the companion papers; here we only need its behaviour. On a development benchmark it is right on all thirty inputs and never wrong without warning. On twenty random equations it returns the right or an equivalent equation on nineteen. Yet on a larger simulated corpus of random equations from five families, on the earlier pipeline version the layers were built on and with every learned layer off, it is *false-confident* on 9 of 80 held-out grid cases: it reports FIT-COMPATIBLE for a wrong factor set.

That figure is the motivation. Is there a learned component, trained on simulated equations, that removes such errors, or that tells us when they occur? The question matters beyond this

*flaflapr@gmail.com

pipeline, because learned correction layers are the usual answer to residual miscalibration in scientific methods, and rarely come with a pre-registered gate and a report of the layers that failed it.

What we did We defined a contract for learned layers (Section 4), built the layers the pipeline admits (Section 2), and evaluated each against a gate fixed before the run, on corpora of simulated equations that do not overlap the development benchmark (Section 5).

What we found Of eight layers (seven built), one is on by default and it is a pure speed-up (Section 8). A learned failure detector is the only layer with a positive effect on the quantity it targets, and that effect does not survive the operating point the contract requires (Section 9). The others are negative: a learned skip of optional steps saves at most 0.7% of the time (Section 10.1); calibration of interval widths and of the top-structure probability does not help (Section 10.2); a scan for high wavenumbers never changes an answer (Section 10.3); a controller that chooses reruns has an upper bound that no available trigger reaches (Section 10.4). Table 5 lists all verdicts. Section 11 extracts what the failures have in common, and Section 12 lists ideas not yet tried, including the author's own.

Structure Sections 2 and 3 describe the pipeline and the statistical learning already inside it; Sections 4 to 6 give the contract, the protocol and the prerequisites; Section 7 states every layer as designed, built or not; Sections 8 to 10 report what was built and what happened, including the controller in detail (Section 10.5) and what was never built (Section 10.6); the appendices hold the failure-mode table, the acceptance protocol and the test inventory.

Scope Everything here concerns linear, constant-coefficient, autonomous, scalar equations in one space and one time variable. The layers were developed on one version of the pipeline (Section 13); we say so wherever it matters.

2 The pipeline and where learning can enter

Figure 1 shows the pipeline as a chain of five steps and the places where a learned layer can attach.

Learning already inside the pipeline Three components are statistical learning and are part of the pipeline, not optional layers: a Gaussian process (with scalable variants) that interpolates scattered data into samples of the field [4]; a Dirichlet-process mixture over the factors of the posterior samples; and a Bayesian linear model that splits the field into one component per factor. They are described and benchmarked in the companion papers. They are fixed here.

The layers Optional layers attach at the places in Fig. 1. L0 proposes starting values for the exponential fit that extracts modes. H0 scans the raw residual for wavenumbers the Gaussian-process path misses. LB caches repeated mode extractions (exact) or learns to skip optional steps. L2b, L2c and L2d calibrate, respectively, the probability of the top structure, the interval widths, and the global constants of the decision rules. V1 predicts from the pipeline's

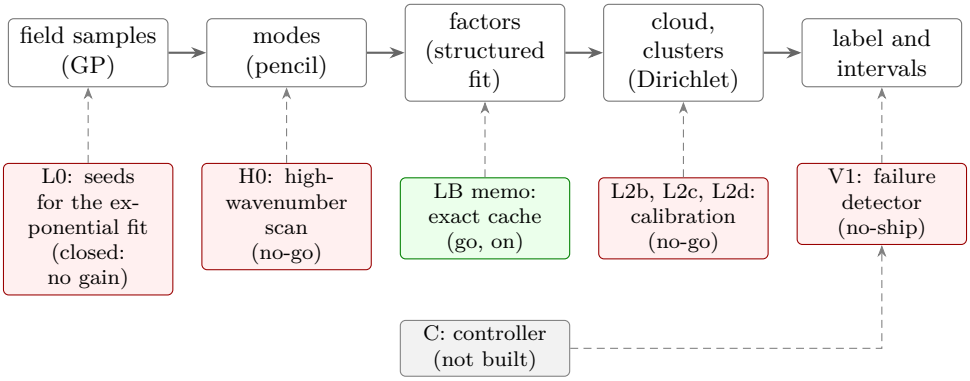


Figure 1: The pipeline (top) and the learned layers attached to it (below). Green: shipped; red: gated and rejected; grey: designed, not built. Dashed arrows show the step each layer touches. V1 only reports; C would have used V1 as its trigger.

own output whether the answer is wrong. C would read V1 and rerun the pipeline with other settings.

3 Machine learning inside the pipeline

Before the optional layers, four components of the pipeline are themselves statistical learning. They are committed, tested and benchmarked, and the optional layers of Section 7 sit on top of them.

3.1 The Gaussian process: interpolation with probabilities

The core needs a field on a grid with an error model, and the data are scattered and noisy. A Gaussian process with a sum of separable kernels $k_t \otimes k_x$ (optionally with a time envelope that mimics $e^{\lambda t}$) gives many plausible fields $\hat{u}_s$ with weights π_s instead of one smooth curve [4]. Hyperparameters, noise included, are marginalised: type-II maximum a posteriori, a Laplace approximation and importance reweighting by default, NUTS for small samples, a random subset for large ones; samples come from pathwise conditioning (Matheron's rule). Alongside it: a factor-wise EPGP (features $e^{\lambda(\kappa)t+i\kappa x}$ on the factors' varieties, closed-form evidence), KISS-GP on a grid (experimental, not calibrated), a deep ensemble (ablation, known to be under-dispersed away from the data), and a bridge that runs the core on the samples. *Tests* (11 in `test_interp.py`): shapes and weights, loose coverage, samples matching the marginals, the subset path for large n, the mixture probability-integral transform against SciPy, the core bridge on transport plus diffusion, EPGP prior draws solving transport, EPGP features on the variety, ensemble smoke test. *Coverage experiment*: nine named cases, $n \in \{400, 2000\}$, noise 1% and 5%, 54 fits; the 90% band covers $0.93, 0.92, 0.90, 0.89$ in the four cells and the median estimated noise is within 4% of the true one; the criterion (± 5 points at $50, 90, 95\%$) holds in 11 of 12 cells, the miss is conservative. An RBF-only family set fails at $n = 2000$ (coverage 0.84–0.86): evidence prefers the smoothest kernel that fits and is overconfident, so the *set* of kernel families is a prior choice made by calibration. The first end-to-end test gave only 0.40 weight to the right factors because the core assumed white noise; this led to per-bin correlated

noise in the core.

3.2 The Dirichlet-process mixture and clustering

The factors of all posterior samples form a weighted cloud W. A truncated stick-breaking Dirichlet-process mixture is fitted per τ-degree (scikit-learn's Bayesian Gaussian mixture) with a merge step by BIC for clusters that never co-occur in a sample; the concentration is chosen by the variational bound and the effective number of factors is k_{eff} = exp of the entropy of the masses. A second module extracts analytic window features and clusters rectangles by four methods (Gaussian mixture, k-means, spectral with adjacency, agglomerative with connectivity), the number of groups being the MAP of a BIC posterior; a dominant-balance baseline is included. *Tests* (11 in `test_aggregate_cluster.py`): invariance to padding and to factor order, aggregates of transport plus diffusion and of heat, rectangles and adjacency, the two-regime ARI for each method, the dominant-balance baseline. *Results*: $P(k = 2) = 1.0$ on transport plus diffusion with masses $(0.768, 0.232)$; $P(k = 1) = 1.0$ on heat after the merge step (margin 7.96 against 8.32, thin); ARI = 1.0 for all four methods when the windows are aligned with the regime boundary and 0.63 for three of them when one straddles it; dominant balance 0.20–0.35. These runs used the exact components as input, so they show the method on clean separation only.

3.3 The Bayesian split layer

A linear-Gaussian model per Fourier bin gives credible intervals for the component weights (by Monte Carlo) and the probability that the split exists, with a real (conjugate-pair) parametrisation at $\kappa = 0$ and a per-bin correlated noise covariance from the interpolation draws. The same machinery supplies the *raw evidence*, the Gaussian evidence of the raw scattered observations in the span of the factors' kernels, which the loop uses to choose the structure. *Tests* (10 in `test_phase3.py`): per-bin noise recovers an AR(1) autocovariance, generalised least squares with white draws equals the white core, the weights cover the truth and the null stays low, the decider rejects constant factors per κ, rejects a symbol that interpolates few modes, counts only observed branches, rejects an interpolation artefact, and merges conjugate pairs. *Experiment*: noise 10^{-2}, true factors; transport plus diffusion: 90% interval coverage 0.95–1.00 for $T \in [0.25, 2]$; wave 0.85–1.00; null (heat with a spurious transport) $P(\text{split}) = 0.000$. The first version covered only 0.65–0.85, which led to the conjugate-pair parametrisation.

3.4 The decider, the refinement and the loop

The loop chooses the structure by raw-data evidence with an Occam term and an outlier-bin mixture, refits the coefficients to the observations by variable projection with terms selected by evidence minus $\frac{1}{2}p \log n$, splits reducible quadratics and conjugate pairs, and grows the degree only under a held-out guard. *Benchmark*: ten named cases $\times$ noise $\{10^{-3}, 10^{-2}, 5{\cdot}10^{-2}\}\times$ sampling {grid, 2000, 400}, 30 runs per cell, against PDE-FIND, weak SINDy, ensemble SINDy, Bayesian SINDy and BOP-DMD: right factor set on a grid 1.00/1.00/0.93, with 2000 points 1.00/0.97/0.90, with 400 points 1.00/0.93/0.73, at least as good as every baseline in all nine cells; coefficient cover$_{90}$ 0.75–0.93, weight cover$_{90}$ 0.50–1.00. *Held-out equations*: random equations by family, 13 draws, right or equivalent on 10, one correct abstention, no false-confident answer. *What this part does not show*: the ten named cases both tuned and measured the

defaults; the hand defaults ($P_{\text{conc}} = 0.9$, γ_N floor 0.02, SNR cut 3, outlier cap $3s$) are still hand-set, and the weight intervals are the least calibrated output.

4 The contract for a learned layer

1. **Off is today, bit for bit.** With every layer off the output equals the pipeline's output exactly: factor strings, labels, intervals, evidence. This is a test, on stored outputs, not a promise.
2. **Propose, calibrate or flag; never decide.** A layer may output a candidate (a mode, a structure) that the pipeline's own evidence then scores; a global constant or scale fitted offline; or a report flag. Labels come only from the deterministic evidence and the identifiability rules.
3. **Fixed likelihood.** Nothing learned changes the noise model or the Occam penalty for a run.
4. **Monte Carlo, not linearisation.** Interval scalings act on the Monte Carlo draws themselves.
5. **A gate before switch-on.** A layer is switched on only if it passes held-out evaluation on equation families it was not trained on (leave-one-family-out), with calibration at least as good as the pipeline's and clean null runs. A layer that fails stays off; its code, its tests and the measured reason stay in the repository.
6. **Scale-free inputs.** Learned features are dimensionless, so that a rescaling of the coordinates does not change a decision.

Rules 1 and 2 are what make a negative result cheap to obtain and safe to keep: a failed layer is switched off and the pipeline is unchanged.

5 Protocol

Corpora Equations are drawn from five simulator families F1 to F5 of random factored operators with exact dispersion relations. Corpus G has grid inputs at 1% and 5% noise; S has scattered inputs; N has nulls (pure noise and single-factor equations); V has 100 clean 5%-noise grid runs (fit seeds 600–607, calibration 608–615, test 616–619) that were added so that a conformal gate at level 0.05 can be evaluated; and I has injected failures.

Injected failures Corpus I has seven types at three strengths: two laws in two time halves, a weak nonlinearity, a variable coefficient, a fractional symbol, coloured noise, heteroscedastic noise, and aliasing. Strengths are scale-free: for deviations of the law, $z = \|u_\varepsilon - u_0\|^2/(\sigma^2\sqrt{2N}) \in \{2, 8, 32\}$; for coloured noise, an AR(1) coefficient in $\{0.3, 0.6, 0.9\}$. Aliasing is a frame shift, $\tau \mapsto \tau + V\xi$, which keeps the factor structure and pushes frequencies beyond the Nyquist limit; a time rescaling would have moved the draw out of the simulator's prior.

Statistics Effects are paired differences with a 95% cluster bootstrap over base draws (seeds), because many injected rows are variants of one draw and a row bootstrap would be too narrow. Validation is leave-one-family-out (LOFO) and, for injection types, leave-one-type-out. Oper-

ating points use split or cross-conformal calibration [2] and conformal risk control [1]. Learned models are gradient-boosted trees [3] or logistic regression on dimensionless features.

Gates A layer ships only if its gate passes in every LOFO fold, not on average. We report "go", "no-go" or "unclear", the last when the sample cannot decide it.

6 Prerequisites: the oracle study, the scoring and the simulator

Three pieces of work with no machine learning come first, because without them a layer that "fixes" 400 points at 5% noise may only be importing its training prior: the current amplitude law makes $\kappa \geq 5$ undetectable whatever the method.

P0.1 Oracle information study Every failing case (case, sampling, noise, seed) is classified on the *raw* observations into information *absent*, *present but lost by the Gaussian process*, or *present but lost by the search*. For each true mode the Fisher information and the Cramér–Rao bound of (λ_j, a_j) are computed at the true values; the raw evidence of the true structure at the true coefficients, and after refinement started at the truth, is compared with the pipeline's answer; the matched-filter signal-to-noise per κ is reported. "Absent" is defined without reference to the pipeline's output, to avoid a circular benchmark: a factor is absent when fewer than $d_x + 1$ distinct κ have a bound on λ below a declared tolerance, branch by branch (transport plus diffusion has one weak branch per κ, so heat's numbers do not carry over).

P0.2 Scoring that rewards honest abstention The benchmark adds *correct abstention* (ID-KER or UNDECIDED where P0.1 says the information is absent) and *false-confident* (FIT-COMPATIBLE with a wrong factor set), and reports risk–coverage curves of selective prediction. Without this every learned layer is pushed toward overconfidence.

P0.3 Simulator extension Ten fixed cases and the seed that only changes phases are not enough. The simulator draws random symbols by family (structure $k \leq 3$, factor degrees $m_j \in \{1, 2\}$, $d_{x,j} \leq 4$, coefficients from a declared prior with a stability constraint, conjugate pairs for real fields, optional shared $\kappa = 0$ modes and repeated factors), random excitation (number and range of κ, amplitude law, T, L, phases), random sampling (grid, 2000, 400, random $n \in [200, 5000]$) and noise (white, AR(1), heteroscedastic, Student-t outliers), and injects failures with labels known by construction (variable coefficient, weak nonlinearity by a split-step solver, fractional symbol, two laws in two halves, a single excited mode, κ above the Nyquist limit, a degree beyond the start bounds, a near-reducible irreducible factor, a reducible product with a nearly square discriminant). Five *families* define leave-one-family-out (LOFO) validation: F1 real first order, F2 dispersive with a complex symbol, F3 second order in τ, F4 products and superpositions with $k \geq 2$, F5 edge cases (shared $\kappa = 0$, repeated factors, single mode). Held-out axes also include a degree, the noise colour and the density. The code is `simulator.py` (every draw has an exact dispersion relation, tested) and the injection generators in `benchmarks/ml_layers/ll_inject.py`.

Table 1: The layers of the plan: verdict of the critical evaluation (before building) and what happened.

layer	role	plan verdict	outcome
P0	oracle study, scoring, simulator	adopt, first	built (simulator, scoring); oracle study partial
L0	learned seeds for the per-bin exponential fit	adopt as a proposal	closed: true poles as seeds give no gain
L1	high-κ correction after the Gaussian process	modify: L1a deterministic, L1b learned only if needed	L1a built as H0, no-go; L1b not built
L2	calibration of probabilities, intervals, constants	modify: fewer degrees of freedom	L2b, L2c, L2d built, all no-go
V	validator	modify: V0 deterministic battery, V1 learned combiner	V0.1 etc. in the pipeline; V1 built, no-ship
C	controller	modify: rule table with an honesty rule	built (C0, C1 and the machinery), not shipped; C′ not built
L5	learned grouping encoder	reject now, defer	not built
L6	amortised posterior (NPE)	defer, cross-check only	not built
LB	exact memo, learned skip	(added during the work)	memo: go; skip: no-go

7 The layers as designed

This section states every layer of the plan with its reasoning, including the layers that were never built. Section 10 and Sections 8 and 9 give what happened to the ones that were. The central position of the plan is one sentence:

> *Machine learning may change where the Bayesian algorithm looks and how fast it gets there, and it may calibrate the prior on a declared simulator distribution. It may never change what the data say about a candidate.*

Every learned output is therefore a *proposal* (candidate modes, factors, structures, initial values, interpolation priors) scored afterwards by the exact raw-data evidence with its Occam term; a *calibration* of a prior or constant fitted offline, checked by simulation-based calibration and reported as prior-dependent; or a *diagnostic flag*. A proposal works like an enumeration aid: it decides which candidates are scored, not how they score. Correct importance weights would need the proposal density after refinement, which is not available, so no importance-sampling exactness is claimed; the claim is weaker and tested by the null gates: a candidate's score does not depend on who proposed it.

7.1 L0: learned seeds for the exponential fit

The matrix pencil is biased at low signal-to-noise and its order choice fails when exponentials merge (an open problem of the theory: Prony and Hankel conditioning has no theory). A learned estimator could give better *seeds* (order and λ) for the existing BIC order choice and variable-projection refinement. *Design*: a one-dimensional CNN or small transformer over the complex series of one κ-bin with a set-valued output, either a pseudo-spectrum head on a fine complex-λ grid (in the style of DeepFreq for undamped lines, extended to damping) or an order classifier plus top-K candidates. *Training data*: generic damped exponential sums with random order $r \leq 8$, near-merged pairs ($\Delta\lambda$ down to $0.1/T$), amplitude ratios down to 10^{-3}, white and coloured noise at SNR 1 to 10^4; these are not PDE solutions, since every bin of every in-scope field is such a sum, so the model never sees a PDE family. *Guards*: seeds never bypass the order criterion; the null gate (no extra modes on single-mode and pure-noise bins). *Outcome*: closed. Even the true poles as seeds do not improve the per-bin step, because the estimator is already efficient and its misses are information-limited.

7.2 L1: high-wavenumber correction

The proposal was "raw data and interpolant in, the missing high-κ parts out" from a network. The plan's objections: the known part needs no learning (the posterior mean of a stationary-kernel Gaussian process is asymptotically a filter, and undoing it is either unnecessary or an amplification of noise); the unknown part is prior (whatever the network adds beyond what the raw data support comes from its training distribution, and neural interpolants have more spectral bias than a Gaussian process); the Gaussian process is the wrong input (the raw observations, not the GP mean, carry the evidence); and the real cause is prior mismatch (a separable stationary kernel gives a fast decay e^{-10t} almost no prior mass). *L1a, deterministic*: (i) a raw-residual mode search, a matched filter for damped exponentials over a (κ, λ) grid on the residual of the current structure, peaks refined by variable projection and offered as candidates, with the acceptance threshold taken from the null distribution of the *maximum* of the scan (the Davies look-elsewhere problem); (ii) a variety-aware interpolation prior (EPGP with learnable spectral points, or spectral-mixture kernels); (iii) a Kronecker Gaussian process on grids so that grids also give posterior samples. *L1b, learned proposal*, only if L1a still leaves information present but missed: a set encoder over the raw residual points with a mixture-density head over (λ, a) per κ, trained on 3 to 5 thousand pipeline residuals with LOFO. Rejected alternatives for this role: Fourier neural operators and DeepONets (forward surrogates for a known PDE), PDE foundation models (gridded fluids, no calibrated uncertainty), scattered-input reconstructors (trained per family; they reconstruct the prior), diffusion priors (circular for identification), SIREN and Fourier features (no uncertainty). *Outcome*: L1a(i) was built as the H0 scan (Section 10.3), no-go; L1a(ii),(iii) and L1b were not built.

7.3 L2: calibration

About 60 method constants exist; one evaluation on a 300-run subset costs about an hour on six workers, so 30 to 50 Bayesian-optimisation evaluations are affordable, enough for four or five constants. The constants are of three kinds: likelihood-defining (they change what the evidence *is*), decision thresholds (they act on probabilities and should follow from a stated loss once the probabilities are calibrated) and computational settings (more is better, subject to cost). Only

the first kind needs data, and only globally. No method can guarantee extrapolation beyond the training family: conformal guarantees hold under exchangeability with the calibration set, i.e. within the simulator prior. *L2a* is diagnostics without learning (simulation-based calibration and TARP coverage per family, reliability of the structure posterior). *L2b* calibrates the structure probability with a g-prior on the coefficients and a structure prior $\log \pi = -\beta_1 k - \beta_2 \sum(m_j + d_{x,j})$ plus an isotonic map, and derives each decision threshold from a declared loss. *L2c* recalibrates intervals with one monotone inflation map per quantity type on three to five dimensionless theory features (the split amplification $\gamma_N^{-1/2}\sigma_n/|w|$, the mode-count margin, the Vandermonde condition, the per-factor SNR, the posterior disagreement), with Mondrian conformal coverage on top. *L2d* tunes at most four or five likelihood-defining constants once and globally by multi-objective Bayesian optimisation. *Outcomes*: L2b no-go (saturated input), L2c no-go (over-covers once the defect it repaired was fixed), L2d no-go (every fold picks today's values); L2a was run as reliability diagnostics.

7.4 V: the validator

The proposal was a network that takes all outputs of every previous iteration and returns $P(\text{valid})$, a diagnosis and decisions for the next loop. The objections: $P(\text{valid})$ is prior-relative and the most important failures (nonlinear, variable coefficient, fractional) are exactly the out-of-distribution inputs; "all outputs" is too many inputs for a few thousand labelled runs; most diagnoses are computable deterministically as posterior-predictive checks with a known null; and diagnosis and action must be separate, since a network that does both cannot be tested component by component and changes the likelihood per run. The re-simulation idea is right: simulate the fitted PDE with the fitted amplitudes at the same design and noise and rerun the statistic, which gives the null distribution of every deterministic test.

V0.1 (with its smooth variants), V0.2, V0.3, V0.10 and V0.13 are implemented in `validate.py`; the others were only specified (V0.6 needs a noise sweep). *V1* is a learned multi-label combiner of the V0 vector (gradient-boosted trees, one-vs-rest per failure mode, with an out-of-distribution score from conformal outlier p-values and a Mahalanobis distance, and conformal risk control for the set of reported failures); Section 9 gives its result.

7.5 C: the meta-controller

The proposal: the validator decides whether to loop again and how, instead of looping blindly. The idea of a *targeted* loop is right (the pipeline already loops in a fixed order: degree growth, regroup, complex rerun, alternatives). The objections are the garden of forking paths, the cost of online optimisation, and the rejected learned encoder. If the controller changes the likelihood (the cap, the signal-to-noise cut, the noise model, the Occam weight) until the validator passes, the final evidence is the maximum over a search and its probabilities are inflated; about ten actions per run at 10 seconds to 3 minutes each leave no budget for per-dataset optimisation; a reinforcement-learning policy needs orders of magnitude more episodes and gives no reasons. *The honesty rule* is the key rule of Phase 4. The controller may change only (a) the *candidate set* (proposals, higher degrees, split and merge moves, complex rerun, alternatives), (b) *computational settings* that do not change the target (more samples, restarts, more alternatives, Monte Carlo size), (c) the *interpolation prior used to generate proposals*. It may never change likelihood-defining constants or the structure prior per run. Two effects remain even then (renormalising over the *visited* set drops the mass of unvisited structures,

Table 2: V0: the deterministic diagnostic battery (each statistic has a parametric-bootstrap null; the thirteen are combined into one omnibus statistic).

id	statistic	detects
V0.1	raw-residual energy and whiteness per κ band	high-κ loss, missing factor
V0.2	mode-count margin $n_\kappa - (d_x + 1)$ per factor	ID-KER, interpolating factor
V0.3	evidence gap to the second structure and to "$\pm$ one factor"; posterior entropy	unresolved ambiguity, spurious factor
V0.4	per-sample disagreement of the raw evidence	local optimum, multimodal posterior
V0.5	Ljung–Box and spectral flatness of the residual in t, variogram in x	correlated noise
V0.6	trend of the selected degree across a noise sweep	out of class (fractional, non-polynomial symbol)
V0.7	$\lambda(\kappa)$ drift across t-windows and subdomains	variable coefficients, mixed regimes
V0.8	amplitude scaling and κ-coupling test	nonlinearity
V0.9	discriminant-square gap, split-quadratic test	unsplit or near-reducible factor
V0.10	c_F, γ_N, $\gamma_N^{-1/2}\sigma_n$ against the weight	weights inseparable at this noise
V0.11	confluent test $(A + Bt)e^{\lambda t}$	repeated factor
V0.12	classifier two-sample test, observed vs. re-simulated residual field	any remaining misfit
V0.13	held-out-in-time score	overfit

Table 3: Diagnosis → remedies (the plan's table; the built controller extends it to thirteen remedies, Section 10.5).

diagnosis (V0/V1)	action
high-κ loss (V0.1)	residual scan, then seeds, then variety prior, then learned proposal; stop with ID-KER if V0.2 fails
degree too low (V0.6 saturating)	the existing degree growth
out of class (V0.6 non-saturating, V0.7, V0.8)	**stop**: MISFIT or UNDECIDED with the reason and a subdomain report
local optimum, disagreement (V0.4)	more initialisations and samples, more alternatives
spurious factor (V0.3)	refine and drop (exists)
unsplit reducible (V0.9)	split quadratics
near-reducible irreducible (V0.9), weights inseparable (V0.10)	none: keep and report the floor
correlated noise (V0.5)	none extra: coloured-noise candidates are always in the declared space
repeated factor (V0.11)	stop: UNDECIDED

and more restarts give a better optimum and a higher Laplace score), so the plan requires: (1) the candidate space declared before the run, with the structure prior split evenly within each complexity class and an upper bound on the unvisited mass reported; (2) an equal refinement budget per candidate, fixed in advance; (3) both noise models (white and coloured) as candidates from the start; (4) the *forking-paths test*, on null data the rate of false FIT-COMPATIBLE with the controller must equal the rate without it. Held-out confirmation is decided *before* the run, by Gaussian data fission or thinning (valid when the noise covariance is known) or by a random point split fixed in advance (valid under any noise, at half the density). At most four actions; no anytime-validity claim. Table 3 is the plan's remedy table; the order of remedies within a diagnosis is learned offline as the success rate per remedy on failure-injected runs (a contextual-bandit estimate with no online exploration). Section 10.5 reports what was built.

7.6 L5, L6 and the differentiable core

L5, a learned grouping encoder was rejected for now. A stable linear encoder that makes the interaction term vanish does not exist and the log-symbol reads terms linearly, so grouping features must be analytic; grouping is by factors only, and today's failures are not grouping failures. A learned metric over factor tracks would merge or split tracks without an evidence statement, hiding the disagreement that the pooling step reports; wrong augmentations merge branches that are close at low κ, exactly where the data are; new factor types are out of distribution and get confident wrong embeddings; and in 2+1 dimensions with non-comaximal factors the decomposition ambiguity is infinite-dimensional, so an encoder would choose a decomposition arbitrarily. It could help later for variable coefficients through frozen-coefficient cells and for 2+1 with comaximal factors, but even there the baseline is deterministic Hungarian matching with a Mahalanobis cost, and a learned similarity would only supply the cost matrix, judged by the adjusted Rand index against injected regimes. **L6, an amortised pos-**

Table 4: Ordered build plan; each step has a gate, and a step that fails its gate stays off.

#	step	gate
1	P0.3 simulator extension	exact dispersion test of every draw
2	P0.1 oracle study, P0.2 scoring	every failure classified
3	L2a calibration diagnostics	report only
4	V0 battery with re-simulation null	$\pm 5\%$ false-alarm rate on correct fits
5	L1a(i) residual search, (iii) grid GP	lost cases recovered, nulls clean
6	C controller with honesty rule and fixed order	forking-paths null test
7	L1a(ii) variety prior	right set up at 400 points, coverage unchanged
8	L2b calibrated Occam prior, loss-derived thresholds	ECE down, LOFO
9	L2c monotone and conformal intervals	coverage $\pm 5\%$ per family on LOFO
10	L2d global optimisation of ≤ 5 constants	LOFO not worse
11	V1 combiner, out-of-distribution score, remedy order	fewer actions at equal accuracy
12	L0 learned seeds	efficiency vs. the bound; benchmark $\geq$ off
13	L1b learned residual proposal	LOFO, nulls clean
14	L6 posterior cross-check	simulation-based calibration
15	L5 encoder	only with 2+1 or variable coefficients in scope

terior (neural posterior estimation) was deferred and demoted to a cross-check validated by simulation-based calibration: agreement is reported, disagreement raises a flag tagged prior-dependent, and it is never a label; its information is already covered by V0 and V1 at a fraction of the data ($\geq 10^4$ runs would be needed). **A differentiable core** is not needed: refinement already optimises the raw evidence over the coefficients, which is the end-to-end step the original note asked for, without touching the core.

7.7 Build order

Steps 1, 3–12 were taken in part (Section 10); steps 13–15 were not.

8 What worked: an exact speed-up

The mode extraction and the confluence test inside the core depend only on the sample, the noise and the mode options, not on the structure being tried. Within one run the pipeline calls the core repeatedly on the same sample (growth, refinement, re-runs), so a memo keyed on these inputs returns identical results. Profiling found the confluence test at 80–90% of core time.

The memo is exact by construction and by test: 0 of 47 paired held-out runs, 10 null pairs,

10 named cases and 10 timing pairs differ in any label, factor string, coefficient or weight. On the pipeline version of the first measurement the total runtime ratio was 0.928 [0.896, 0.948]. After the pipeline's own later speed-ups (a faster confluence test) the same measurement gives 0.995 [0.992, 1.000]: the memo is now worth about half a percent. It stays on because it is exact and harmless, and because its cost is a few lines. This is the only layer that ships, and its lesson is also a warning: a speed-up measured against a slow baseline can disappear when the baseline improves.

9 The failure detector V1

V1 is a gradient-boosted classifier that takes dimensionless features of the pipeline's output (labels, evidence margins, residual statistics) and predicts whether the answer is wrong. It is report-only: it changes no label. A variant would downgrade a flagged FIT-COMPATIBLE to UNDECIDED.

Feasibility On the first pipeline version, over 49 clean and 92 scored injected runs, LOFO AUROC is 0.900 [0.825, 0.975] against 0.681 for the pipeline's own omnibus residual test (difference +0.219 [+0.078, +0.289]); on FIT-COMPATIBLE runs only, where an undetected failure matters most, 0.759 against 0.515, which is chance. On fresh base draws the estimate is about 0.85, because one seed supplies 21 of the 28 injected rows per family and those rows score higher.

Re-gate on the final pipeline With 100 clean 5% grid runs and corpus I re-run, LOFO AUROC is 0.847 [0.720, 0.914] and exceeds the omnibus test by 0.222 [0.018, 0.356]. But the conformal operating point at $\alpha = 0.05$ fails: the false-alarm rate is inside the bound (split 0/17, LOFO 0.00–0.09), and detection is 0.00–0.08 per fold, below the omnibus test's 0.26–0.47. The score separates on average but not in the far tail that a 5% false-alarm level demands. The downgrade variant, replayed on held-out rows, leaves false-confident answers at $7 \to 7$ on clean rows (losing two correct answers) and $3 \to 2$ on injected ones. **Verdict: no-ship.**

Why V1 reads the pipeline's output, so it cannot flag a run that crashes or times out (48 of 140 injected runs on the first pipeline version); it detects none of the injected nonlinear, variable-coefficient or two-law failures on the final pipeline; and an output-only detector inherits the blind spots of the pipeline that produced the output.

10 Layers that did not help

10.1 Learned skipping of optional steps (LB)

Optional steps (grow, recurse, complex, alternatives) can be skipped when a classifier predicts that they leave the state unchanged. A perfect classifier would save 17% of total time on the fitting and calibration runs (14% on test). The zero-harm calibration, cross-fitted over 165 runs, saves 0.2% (harmed runs 0/165); a 2%-risk profile saves 0.7% with 2/165 harmed. On the test split neither skips anything. The largest guarded step changes the state in 64 of 133 runs, and most time is spent in steps that change the state every time and are not optional.

10.2 Calibration: P(top), interval widths, constants

L2b, probability of the top structure The raw probability is at least 0.999 on 96% of rows, so it has almost no resolution; a calibrator that sees only it learns a base rate, and base rates differ by family (right-or-equivalent: 1.00 on F1, 0.77 on F2, 0.82 on F3, 0.36 on F4, 0.75 on F5). Best Brier change for right-or-equivalent is -0.032 [-0.073, $+0.007$], and every arm is worse than raw on F1 where raw is already perfect. *No-go.*

L2c, interval widths An earlier experiment on grids with a stand-in posterior moved coefficient coverage from 0.756 to 0.923 on held-out families. On the final pipeline and 114 test cells the pipeline's own 90% intervals cover 0.895; a scalar rescaling model gives 0.965 and a gradient-boosted model 0.982, wider intervals that over-cover, and neither lies in $[0.85, 0.95]$ on every fold. The earlier gain was a repair of a defect the pipeline had since fixed. *No-go.*

L2d, global constants Tuning the evidence thresholds and the minimum signal-to-noise by Bayesian optimisation chooses today's values in every LOFO fold (change +0.000 [0, 0] for the signal-to-noise cut). *No-go.*

10.3 High-wavenumber scan (H0)

A raw-data matched filter followed by variable projection reaches the Cramér–Rao bound at wavenumber $\kappa \in [1, 4]$ on heat data (400 points, 5%) and recovers $\kappa = 4$ that the Gaussian-process path misses; beyond $\kappa = 5$ the amplitude is below 0.4σ and no method can. Inside the pipeline the scan gave identical labels and factors on 80/80 test pairs, 40/40 nulls and 22/22 named cases (one timeout difference among 30 scattered runs) and only added time: the cases it can help with end as ID-KER or UNDECIDED anyway. *No-go.* L0, seeds for the exponential fit, was closed earlier: even the true poles as starting values do not improve the per-bin step, because the estimator is already efficient and its misses are limited by information, not by initialisation.

10.4 The controller (C)

A controller would read a diagnosis and rerun the pipeline with one of thirteen alternative settings. We measured the ceiling first. On the final pipeline we took every off failure that did not time out in corpus V and its matched control (6 false-confident and 2 abstentions) plus four timeouts, and ran fifteen single actions from today's defaults on each (12 cases). The *oracle* ceiling, any action rescues the case, is $8/12 = 67\%$; `lower_degree` rescues 8, `varpro` 3, and the rest at most 2. With V1 as the trigger the ceiling is 0/12: V1 flags none of the failures, and it raises 3 false alarms on 86 cases the pipeline gets right. The action space has room; *a trigger with resolution on false-confident runs is the open problem.* The controller was not built. An earlier controller version passed forking-path null tests at every depth but had an oracle ceiling of 1/13.

10.5 The controller in detail: what was designed, built and tried

Control space The session replaced the three-action sketch of the plan by a typed `RunConfig` threaded through the pipeline, with every default equal to today's literal (a test checks this

Table 5: Verdicts on all layers. "Gate" is the pre-set criterion; numbers in brackets are 95% bootstrap intervals.

Layer	Does	Key evidence	Verdict
LB memo	exact cache of mode extraction	0 harmed in 77 pairs; time ratio 0.995 [0.992, 1.000]	go, on
LB skip	skip optional steps	saves 0.2–0.7%; skips nothing on test	no-go
V1	flags likely-wrong answers	AUROC 0.847; detection 0.00–0.08 at $\alpha = 0.05$	no-ship
L2b	calibrates P(top)	raw ≥ 0.999 on 96% of rows; ΔBrier -0.032 [-0.073, $+0.007$]	no-go
L2c	rescales intervals	over-covers (0.965, 0.982 vs 0.895)	no-go
L2d	global constants	every fold picks today's values	no-go
H0	high-κ scan	identical output on 80/80, 40/40, 22/22	no-go
L0	seeds for the fit	true poles as seeds: no gain	closed
C	reruns with other settings	oracle 8/12; with V1 trigger 0/12	not built

against the modules, and that `run_loop(config=RunConfig())` equals `run_loop(config=None)` bit for bit on the six golden draws). Knobs are of three classes: *frozen per case* (the likelihood- and label-defining constants: the concentration threshold, the cap, the predictive-p threshold, the support z, the Occam term, the confluent BIC, and so on); *global*, owned by L2d and the memo layer (Monte Carlo sizes whose right value does not depend on the equation: the controller may raise one as a remedy for a measured symptom, never lower it); and *per-case*, owned by the controller: the number of Gaussian-process samples, restarts and iterations, the band $\kappa_{\max}$, degree bounds and limits, the maximum mode order, the number of alternatives kept, the Monte Carlo and refinement budgets, the recursion depth, the use of the high-κ scan, the noise estimate.

The thirteen remedies `more_samples`, `more_restarts`, `wide_alternatives`, `highk_scan`, `narrow_band`, `lower_degree`, `raise_limit`, `max_order_up`, `varpro`, `more_mc`, `refine_budget`, `deeper_recursion`, `sigma_reest`; each is a discrete, cumulative configuration delta. The default diagnosis table (the baseline arm C0) maps twelve diagnoses to ordered remedies, for example `no_factors` $\rightarrow$ varpro, more samples, lower degree, re-estimate σ; `misfit` $\rightarrow$ more restarts, raise limit, more samples, higher mode order; `idker` $\rightarrow$ high-κ scan, lower degree, more samples; `ambiguous_structure` $\rightarrow$ wide alternatives, more Monte Carlo, refinement budget, more samples; `growth_overfit` $\rightarrow$ lower degree, more samples, narrower band; `weights_inseparable` $\rightarrow$ nothing (report the floor). Diagnoses come from signals the pipeline already reports (the reasons for UNDECIDED, the outlier fraction of MISFIT, the unsupported degree of ID-KER, the V0 statistics, sample disagreement, bounds at the limits, growth rejected by the overfit guard, factors dropped, a widened band, a timeout).

The honesty rule, revisited The build tightened the plan's rule. The frozen scoring (evidence formula, Occam term, structure prior, label thresholds) stays the only label authority; the controller chooses configurations, and a label is always the frozen pipeline's output on a configuration. The *declared space* became the typed configuration plus the nuisance estimates (declaring structures only restricts; declaring the knobs controls the multiplicity); equal budget

is required only in the final comparison; the report is the posterior over all visited structures plus a Good–Turing missing mass N_1/N; and held-out confirmation became *mandatory cross-fitted data fission* whenever the factor set would change: with white Gaussian noise of level $\hat{\sigma}$, draw $Z \sim \mathcal{N}(0, \hat{\sigma}^2 I)$ from a case-derived seed and form $A = y + Z$, $B = y - Z$, two independent copies with the same design; search on one, confirm on the other, swap, and keep a change only if confirmed in both directions, so that a spurious acceptance needs two independent coincidences. The price is half the information per fold; the final run uses all data; $\hat{\sigma}$ comes from structure-free estimators (the input, the Gaussian-process noise posterior, or the high-band robust scale on a grid) before the split, and the residual dependence is checked empirically by the null test. High noise is handled before any search: if every factor of the first run has negative identifiability margin, or no mode is above the noise, the controller stops and returns the first run with the reason in the note; it never relabels.

Policy arms C0 rules (the default table, fixed depth, no learning); C1, a table of success rates with a Beta lower credible bound per (diagnosis, remedy) per unit cost and an index stopping rule; C2, a per-remedy logistic success model on dimensionless features with isotonic calibration and a Weitzman-style index stop, explored offline by Thompson sampling and deployed greedily on the 10% lower bound; C3, Bayesian optimisation of the continuous knobs (live, on a subset); C4, offline reinforcement learning (only if C3 beat C2 by a pre-registered margin). Training logs are one-step logs (each applicable remedy plus one random non-indicated remedy with recorded propensity) and multi-step logs (random sequences of three to five remedies), with inverse-propensity and doubly robust estimates of policy value before any live run, and null logs that remove any remedy producing a false FIT-COMPATIBLE on nulls.

The gate On the test split with leave-one-family-out policies: right-set up with a paired bootstrap interval excluding zero; right-set at least off minus 0.02 and false-confident no higher; coverage not worse by more than 0.02; **the forking-paths null test at every loop depth** $0, \ldots, 20$ (the would-be output after each loop with the stopping rule disabled); named cases not worse; reports of runtime and of actions per run; an independent reviewer pass. If no arm passes, the layer stays off.

What the data said *Null test*: on the null corpus (pure noise and single-factor equations) the false FIT-COMPATIBLE rate with the controller equals the rate without it at every depth 0–12, with and without fission; the null test on the injected-failure corpus stopped early (6 of 35 cases of a stratified subset ran and only one was usable, so it is no evidence either way). *First gate*: on the first pipeline version, C0 on the grid corpus had 16 completed cases and changed none (+0.000 [0, 0]), with one timeout at the six-hour cap and one failure in the first run, and the first oracle ceiling was 1/13 off-failures (8%, a lower bound, since only 1.9 of 13 remedies were logged per case); the learning curve of C1 was flat, so more data did not help. *C' oracle sweep on the final pipeline*: the ceiling is 8/12 failures with the best remedy (`lower_degree`: 8, `varpro`: 3, `wide_alternatives`, `narrow_band`, `skip_grow`, `sigma_reest`: 2 each, `skip_recurse`: 1), but with V1, the omnibus test and the other tests as the trigger the ceiling is 0/12 (3/12 if a timeout counts as a trigger). Only 1.9 remedies per case in the first sweep, then fifteen actions per case in the second, show that the action space has room and the *trigger* does not exist. The controller C' (a trigger-driven controller on top of V1) was

therefore not built, by decision on 1 November 2026, and the controller layer stays off. *Tests*: 15 in `test_learned_controller.py` and 10 in `test_runconfig.py`.

10.6 What was designed and never built

- **L1b, the learned residual proposal** (a set encoder over raw residual points with a mixture-density head): only justified if the deterministic scan left information present but missed; the scan changed no answer, so there was nothing to amortise.
- **L1a(ii) and (iii), the variety-aware prior and the Kronecker grid Gaussian process:** the EPGP and KISS-GP code exists in the pipeline but was not chained into the loop; the scan's null result removed the motivation.
- **The remaining V0 statistics** (V0.4 sample disagreement, V0.5 noise whiteness, V0.6 degree trend over a noise sweep, V0.7 and V0.8 drift and coupling tests, V0.9 and V0.11 as p-values, V0.12 classifier two-sample test) were specified in the plan; the pipeline has V0.1, V0.2, V0.3, V0.10 and V0.13, and the effects behind V0.9 and V0.11 (split quadratics, the confluent test) as pipeline steps.
- **L5, the learned grouping encoder:** rejected for now with reasons (Section 7); it becomes relevant only with variable coefficients through frozen-coefficient cells, or 2+1 with comaximal factors.
- **L6, the amortised posterior:** deferred; needs $\geq 10^4$ labelled runs and would only be a prior-dependent cross-check.
- **The differentiable core:** not needed.
- **C′, a trigger-driven controller:** not built (above).
- **L2b with loss-derived thresholds** was built as a calibrator of the top-structure probability only; the decision thresholds were not re-derived.

11 Lessons

Average separation is not tail detection V1 has a good AUROC and almost no detection at a 5% false-alarm rate. A gate at a fixed operating point measures something different from a ranking metric, and the second can look good while the first fails.

A calibrator needs an input with resolution Both probability calibrators see a saturated input. A map from a saturated number can only learn a base rate, and base rates differ by family, so the gain appears in the low-base-rate family and the damage in the others.

An oracle ceiling must be measured before building a controller Whether the action space has room (it does: 67%) is a different question from whether anything points at the cases that need it (nothing does: 0%). Measuring the ceiling cost 180 runs; building the controller would have cost far more.

Baselines move A speed-up of 7% became 0.5% when the pipeline improved; an interval calibration that raised coverage from 0.76 to 0.92 over-covered once the defect it repaired was fixed. Every gate has to be re-run on the final pipeline, which is why the contract keeps failed layers in the repository.

Draw variance dominates small corpora One base draw supplied 21 of 28 injected rows and inflated the headline AUROC from 0.85 to 0.96; cluster bootstrap and fresh draws are not optional.

Crashes and timeouts are outside any output-based detector A detector that reads the output cannot see runs that produce none; 48 of 140 injected runs were of this kind.

12 Ideas not yet tried

These are not results. They are directions the evidence suggests, or that the author proposed during the project.

1. **An input-only validator** that runs before the pipeline and predicts crashes, timeouts and out-of-class data (aliasing, coloured noise) from the data alone. It would cover the failures V1 cannot see.
2. **A trigger with resolution on false-confident runs.** The controller's ceiling is 67%; any trigger that flags even a third of the false-confident answers would change the verdict. Candidates are residual tests on raw data that are sensitive in the tail, rather than a learned score.
3. **Encoding and grouping.** Prepare the data with a nonlinear embedding (t-SNE or an autoencoder) so that the modes of different factors separate naturally, then group with a mixture model; and ask whether encoding plus grouping drives the interaction term to zero. The method should still work without this step.
4. **Regularisation by the number of factors rather than by coefficient size.** An ℓ_1 penalty on the weights of the expansion instead of ℓ_2 favours few components; whether it removes the interaction term is open.
5. **Port the layers to the current pipeline.** The layers were built on an earlier version of the pipeline (Section 13). Porting their hooks to the current one, which has a better refinement stage and fewer false-confident answers, would test whether the negative verdicts survive.

13 Reproducibility and limits

This branch holds the layers, their tests and the experiment scripts. The pipeline in `src/` is the snapshot the layers were developed and tested on (the main pipeline at commit `eaa8f9c` plus the layers). The current pipeline on branch `main` has later changes in `loop.py` and the structured fit that touch the same lines as the layer hooks, so the port in item 5 of Section 12 is real work and is not done here. The directory `variants/s_interp/` holds the separate interpolation variant of the pipeline (15 source files, 3 tests, 14 analysis scripts and its session notes) that was developed in parallel; `claude/archive/sessions/` holds every session plan and report of the layers, and `benchmarks/ml_layers/` the experiment scripts, tables and logs. All numbers in this paper are from stored reports in `benchmarks/ml_layers/results/`. The off-snapshot tests compare against stored outputs whose last digits depend on the numerical library; the stored outputs were regenerated on Linux (NumPy/SciPy as pinned in `pyproject.toml`) and the test suite passes there. Corpora are simulated; no real data are used, and the findings are about this pipeline and this simulator prior.

14 Conclusion

Under a contract that forbids a learned layer from deciding, the pipeline gains almost nothing from learning on top of it: one exact speed-up, one failure detector that ranks well and flags poorly, and a negative result for every calibration layer. The most valuable outcome is the measurement of where the remaining error lives: false-confident answers that no output-based score points at, and a rerun space that could rescue two thirds of them if a trigger existed. Finding that trigger, not adding a layer, is the next step.

A Failure modes, detectors and fixers

D is a deterministic detector, V1 the learned combiner, OOD the out-of-distribution score, C a controller action; "−" in the fixer column means report only (a theory floor).

phase / failure	symptom	detector	fixer
Data and model class			
nonlinear PDE	κ-coupling, amplitude-dependent λ	V0.8, OOD	− (out of scope): MISFIT with reason
variable coefficients	$\lambda(\kappa)$ drifts across cells or windows	V0.7, V0.6	−; subdomain report
fractional / non-polynomial symbol	evidence rises with degree	V0.6	−: UNDECIDED "no finite degree"
two laws in two regions	per-cell posteriors disagree	V0.7	subdomain report
correlated or non-Gaussian noise	non-white residuals	V0.5	coloured-noise candidates
outliers	heavy tails	V0.5, V0.12	outlier-bin mixture (exists)
too few κ per factor	margin < 0	V0.2	−: ID-KER
κ above Nyquist (aliasing)	modes at wrong κ	V0.1, design check	−: report
information absent	P0.1 oracle	P0.1, V0.1	−: ID-KER/UNDECIDED (correct abstention)
Interpolation			
high-κ shrinkage	residual energy at high κ	V0.1	L1a, L0, L1b
ML-II overconfidence	coverage $<$ nominal	PIT (exists)	Laplace-IS, NUTS (exist)
stationary-kernel mismatch	fast decays absent from samples	V0.1, P0.1	L1a(ii)
Modes and structured fit			
pencil bias at low SNR	λ off by $>$ bound	refine gain	VarPro (exists), L0
wrong order, merged exponentials	outlier modes	BIC per bin	L0 seeds
degree or dictionary too low	structured residual	V0.6, V0.13	degree growth

phase / failure	symptom	detector	fixer
local optimum of the alternating fit	samples disagree	V0.4	restarts, screening
spurious factor	low inclusion, small gap	V0.3	refine and drop
unsplit reducible factor	square discriminant	V0.9	split quadratics
near-reducible irreducible	damped transports vs. telegraph	V0.9	merge candidate (exists)
real field, factor reducible over $\mathbb{C}$	no conjugate pair	V0.3	complex rerun (exists)
repeated factor Q^2	confluent wins	V0.11	−: UNDECIDED
Pooling, decision, split, bootstrap, refinement			
label switching	mixed tracks	Hungarian (exists)	−
DP over-counts k	tiny clusters	finite-k BIC (exists)	−
miscalibrated Occam / prior	reliability off	L2a	L2b
interpolating factor	equations $\leq$ unknowns	V0.2	admissibility rule (exists)
weight undercoverage	SBC ranks	L2a	coefficient draws, L2c
inseparable weights	wide intervals	V0.10	−: report the floor
grid bootstrap overwrites refined intervals	inconsistent	code review	L1a(iii)
support path stops early	lower evidence than full support	V0.3	path rule (local fix)
Output and controller			
confident wrong label	false FIT-COMPATIBLE	P0.2 metric	everything above; abstain
controller search inflates evidence	null false-positive up	forking-paths test	honesty rule, data fission
learned layer out of distribution	OOD score high	V1 OOD	layer output suppressed

B Acceptance protocol common to all layers

1. **Regression**: the switch off reproduces the stored outputs exactly.
2. **LOFO**: train and select on four of the families F1–F5 and test on the fifth (families are defined by factor type; a held-out factor type is also removed from the products of F4, otherwise F4 leaks it); also hold out a degree, the noise colour and the density; report in-family and LOFO side by side; a layer that helps only in-family is rejected.
3. **Nulls**: single-factor data give no extra factor; pure noise gives UNDECIDED or MISFIT; data from a held-out injected failure are not labelled FIT-COMPATIBLE.
4. **Calibration**: simulation-based calibration rank histograms and TARP coverage per family,

coverage within ±5% of nominal, reliability of the structure probabilities.
5. **Benchmark**: right set, exact structure, coefficient error, weight error, correct abstention, false-confident rate and runtime, against off and against the best SINDy variant.
6. **Audit**: an independent fresh-context reviewer before any claim.

C Test inventory

The branch's test suite (all pass; the off-snapshot tests use the golden file regenerated on Linux): learned layers and harness `test_learned.py` (24), `test_learned_inject.py` (42), `test_learned_budget.py` (16), `test_learned_controller.py` (15), `test_learned_off.py` (12), `test_runconfig.py` (10), `test_learned_v1.py` (10), `test_learned_ptop.py` (9), `test_learned_resid.p` (8), `test_learned_layers.py` (8), interface pin `test_ml_contract.py` (5); the pipeline underneath: core (`test_core_exact.py` 20, `test_core_v2.py` 21), general structure (15), aggregation and clustering (11), the split layer and decider (`test_phase3.py` 10), the loop (5, 7 for growth, 6 for grid refinement), domains (8), simulator (4), validation (3), benchmark noise (2).

D Libraries and constants

The plan's research pass catalogued the libraries it would use or rejected: FINUFFT (nonuniform FFT for the residual scan), GPyTorch and its spectral-mixture kernels, scikit-learn (histogram gradient boosting, isotonic regression, mixtures), LightGBM, MAPIE and crepes (conformal prediction and risk control), Ax/BoTorch and SMAC3 (offline constant tuning), sbi and BayesFlow (amortised posteriors and diagnostics), PySINDy and PyDMD (baselines). The method constants of the code (about 60) are classified as likelihood-defining, decision or computational in `claude/archive/pipeline/analysis/PARAMETERS.md` (branch `main`); the plan with its sources is `claude/notes/PHASE4_ML_PLAN.md`.

References

[1] Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. Conformal risk control. In *International Conference on Learning Representations*, 2024. arXiv:2208.02814.

[2] Rina Foygel Barber, Emmanuel J. Candès, Aaditya Ramdas, and Ryan J. Tibshirani. Predictive inference with the jackknife+. *The Annals of Statistics*, 49(1):486–507, 2021.

[3] Jerome H. Friedman. Greedy function approximation: a gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.

[4] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, Cambridge, MA, 2006.